Unit II

SQL and PL/SQL

Syllabus

SQL: Characteristics and Advantages,

SQL Data Types and Literals,

DDL, DML, DCL, TCL,

SQL Operators.

Tables: Creating, Modifying, Deleting, Updating.

SQL DML Queries: SELECT Query and clauses, Index and Sequence in SQL.

Views: Creating, Dropping, Updating using Indexes, Set Operations,

Predicates and Joins, Set membership, Tuple Variables, Set comparison, Ordering of Tuples, *Aggregate Functions*, SQL Functions, Nested Queries.

PL/SQL: Concept of Stored Procedures and Functions, Cursors, Triggers, Assertions, Roles and Privileges.

- What is view and how to create it? Can you update view? If yes, how? If
- not, why not?

SQL: Characteristics and Advantages

❖ **Structured query language**

❖ **Characteristics:**

- SQL is easy to learn.
- SQL is used to access data from relational database management systems.
- SQL can execute queries against the database.
- SQL is used to describe the data.
- SQL is used to define the data in the database and manipulate it when needed.
- SQL is used to create and drop the database and table.
- SQL is used to create a view, stored procedure, function in a database.
- SQL allows users to set permissions on tables, procedures, and views.

❖ **Advantages:**

- High speed
- No coding needed
- Well defined standards
- Portability
- Interactive language
- Multiple data view

DDL: data definition language

- Create database college;
- Use database college;

```
Create table student (
                    Rollno number(50),
                    name varchar(30),
                      class varchar(20)
);
```

Truncate table student;

Rollno	Name	Class

SQL: Characteristics and Advantages

❖ **Advantages:**

- **High speed:**
  - Using the SQL queries, the user can *quickly* and *efficiently retrieve* a *large amount of records from a database.*
- **No coding needed:**
  - In the standard SQL, it is very easy to manage the database system. It *doesn't require a substantial amount of code to manage* the database system.
- **Well defined standards:**
  - Long established are used by the SQL databases that are being used by *ISO and ANSI.*
- **Portability:**
  - SQL can be used in laptop, PCs, server and even some mobile phones.
- **Interactive language:**
  - SQL is a *domain language* used *to communicate with the database*. It is also used to receive answers to the complex questions in seconds.
- **Multiple data view:**
  - Using the SQL language, the users can *make different views of the database structure.*

Data Types

CHAR(size)	It is used to ***specify a fixed length string*** that can contain numbers, letters, and special characters. Its size can be 0 to 255 characters. Default is 1.
VARCHAR(size)	It is used to ***specify a variable length string*** that can contain numbers, letters, and special characters. Its size can be from 0 to 65535 characters.
BINARY(Size)	It is equal to CHAR() but ***stores binary byte strings***. Its size parameter specifies the column length in the bytes. Default is 1.
VARBINARY(Size)	It is equal to VARCHAR() but ***stores binary byte strings***. Its size parameter specifies the maximum column length in bytes.

Data Types

TEXT(Size)	It holds a ***string*** that can contain a maximum length of 255 characters.
MEDIUMTEXT	It holds ***a string*** with a maximum length of 16,777,215.
LONGTEXT	It holds a ***string*** with a maximum length of 4,294,967,295 characters.
BLOB(size)	It is used for ***BLOBs (Binary Large Objects).*** It can hold up to 65,535 bytes.
Int or Number(Size)	for storing numbers

Database Languages

- **SQL Statements/Commands/Language:**

✓ Data Defination Language (DDL)

✓ Data Manipulation Language (DML)

✓ Data Control Language (DCL)

✓ Transaction Control Language (TCL)

Data Definition Language (DDL):

- DDL is an abbreviation of **Data Definition Language.**
- The DDL Commands in SQL are ***used to create and modify the schema or structure of the database and its objects.***
- **Following are the five DDL commands in SQL:**

✓ CREATE Command

✓ DROP Command

✓ ALTER Command

✓ TRUNCATE Command

✓ RENAME Command

Database Languages

Data definition language (DDL)

✓ **CREATE Command:**

- CREATE is a DDL command used to create databases, tables, triggers and other database objects.

- **Database :**

- **CREATE Database** Database_Name;

Ex:

Create Database Books;

Database Languages

Data definition language (DDL)

- ✓ **CREATE Command:**

Table creation:

Syntax:

```
CREATE TABLE table_name
(
        column_Name1 data_type ( size of the column ) ,
        column_Name2 data_type ( size of the column) ,
        column_Name3 data_type ( size of the column) ,
        ...
        column_NameN data_type ( size of the column )
) ;
```

Database Languages

- **Example**:

Create Database Books;

CREATE TABLE Student

(

Roll_No **Number(20)** ,

First_Name **Varchar** (30) ,

Last_Name **Varchar** (20) ,

);

Database Languages

Drop: drops or delete database or table .

Syntax: for delete database

DROP **DATABASE** Database_Name;

Example:

Drop database College;

Syntax: for delete table

DROP **TABLE** Table_Name;

Example: consider table student

Drop table student;

- **TRUNCATE Table Command:** empty the table or remove data from table.

Syntax:

TRUNCATE **TABLE** Table_Name;

Example:

TRUNCATE **TABLE** student

- **RENAME Table Command:**

RENAME **TABLE** Old_Table_Name **TO** New_Table_Name;

Example: consider table student and we wish to change name to stud_info

RENAME **TABLE** student **TO** stud_info;

Database Languages

- **ALTER Command:**

- ALTER is a DDL command which ***changes or modifies the existing structure of the database***, and

- it also changes the schema of database objects.

- We can also **add** and **drop** constraints of the table using the ALTER command.

ALTER TABLE name_of_table **ADD** column_name column_definition;

ALTER TABLE name_of_table **DROP** Column_Name_1 , column_Name_2 , ….., column_Name_N;

ALTER TABLE table_name **MODIFY** (column_name column_datatype(**size**));

Database Languages

Example: adding new column to existing table

ALTER TABLE Student **ADD** Class **Varchar**(30);

Example: drop column from table

ALTER TABLE Student **drop** Age, Marks;

Example: Change the size of column Last_Name:

ALTER TABLE table_name **MODIFY** (Last_Name **varchar**(25));

DML Commands in SQL:

- DML is an abbreviation of **Data Manipulation Language.**
- It ***change the data present in the SQL database.***
- can easily access, store, modify, update and delete the existing records from the database using DML commands.
- **Following are the four main DML commands in SQL:**

- SELECT Command
- INSERT Command
- UPDATE Command
- DELETE Command

Database Languages: DML Commands in SQL

DML Commands in SQL:

Select:

SELECT * **FROM** table_name;

SELECT Emp_Id, Emp_Salary **FROM** Employee;

SELECT * **FROM** Student **WHERE** Stu_Marks = 80;

Insert:

INSERT INTO TABLE_NAME (column_Name1 , column_Name2 , column_Name3 , ... column_NameN) **VALUES** (value_1, value_2, value_3, ... value_N) ;

Update:

UPDATE Table_name **SET** [column_name1= value_1, ..., column_nameN = value_N] **WHERE** CONDITION;

Database Languages: DML Commands in SQL

Select:

- SELECT is the most important data manipulation command in Structured Query Language.
- The SELECT command *shows the records of the specified table.*
- It also *shows the particular record of a particular column* by using the ***WHERE*** clause.
- **Syntax:**

1. SELECT * **FROM** table_name;

2.SELECT column_Name_1, column_Name_2, ….., column_Name_N **FROM** Name_of_table;

3. SELECT * **FROM** table_name where <condition>;

Database Languages: DML Commands in SQL

Examples :

Select:

1. **Select** * **from** student;

2. **Select** Roll_no, Sname **from** student;

3. **Select** * **from** Student **where** Roll_no = 6;

Database Languages: DML Commands in SQL

2. Insert command :

is used to ***insert data*** into a table.

Syntax:

INSERT INTO table_name *VALUES* (data1, data2, ...)

or

INSERT INTO table_name (column1, column2,..) *VALUES* (data1, data2, ...)

Example:

Insert into student(Rollno, Name, Class) values (1, 'Abhay', 'TE');

Database Languages: DML Commands in SQL

Insert NULL value to a column:

Example:

INSERT INTO student(roll_no, name) values(6, 'Abhay');

or

INSERT INTO Student VALUES(6, 'Abhay', null);

3. UPDATE Statement:

The UPDATE statement is used to modify the existing records in a table.

Syntax:

UPDATE Table_name **SET** [column_name1= value_1, ….., column_nameN = value_N]
WHERE CONDITION;

Database Languages: DML Commands in SQL

Example of update:

- Change the name of student to '**Riyansh**' who have a rollno.= 02 in student table

  Update student set **name**='Riyansh' where **RollNo**=02;

- **Delete :**

- Used to delete the whole record or particular record from table.

- **Syntax**:
- Delete from <table_name> where <condition>;

Database Languages: DML Commands in SQL

Delete :

Example:

1. Delete the record from student table.

```
Delete from Student;
```

2. Delete the record of stduent who have a roll no. 2

```
Delete  from student where rollno= 2;
```

Note : for DML commands use any table with values.

DCL: Data Control Language

DCL: Data Control Language:

- Data control language (DCL) is used to ***access the stored data***.
- It is mainly used *for revoke and to grant* the user *required access to a database.*
- It is related to *database security and integrity.*
- It performs a task of *assigning privileges , so* users can access certain objects in the database.
- *We can perform or apply privilages by using following commands*
- *Commands:*

✓ ***Grant***
✓ ***Revoke***

DCL: Data Control Language

- It provides the administrators, to remove and set database permissions to desired users as needed.

- These commands are employed to grant, remove and deny permissions to users for retrieving and manipulating a database.

- **Privileges:**
  - ✓ All
  - ✓ Alter
  - ✓ Delete,
  - ✓ index,
  - ✓ insert,
  - ✓ Update
  - ✓ select and
  - ✓ drop

DCL: Data Control Language

- **a. Grant:**
- It is used to give user access privileges to a database.
- Object created by one user are not accessible by another users unless the user of those objects gives such permissions to other users.
- These permissions can be given by using the grant statement.
- Grant statement provides various types of access to database objects such as tables, views and sequences.
- User can grant all the privileges or grant only specific privileges.

DCL: Data Control Language

- **Syntax:**
- GRANT Privileges ON <Table_Name> TO <USER_Name>;

- **Example 1**: Grant all Privileges on student table to user Atherva
- GRANT All ON Student TO Atherva;

- **Example 2**: Grant Select & update Privileges on student table to user Atherva
- GRANT select, Update ON Student TO Atherva

DCL: Data Control Language

- **b. Revoke:**
- It is used to take back/withdraw/deny the granted permissions from the user.
- REVOKE ***Privileges*** ON <Table_Name> ***from*** <USER_Name>;
- **Example 1:**
- REVOKE **All** ON Student ***From*** Atherva;
- **Example 2:**
- REVOKE **select, Update** ON Student ***From*** Atherva;

TCL: Transaction Control Language

Transaction:

- Single logical unit of work which is the collection of several operations.
- TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.
- These operations are *automatically committed in the database* that's why they cannot be used while creating tables or dropping them.
- Here are some commands that come under TCL:
- COMMIT
- ROLLBACK
- SAVEPOINT

TCL: Transaction Control Language

- **COMMIT:**
- Commit command is used to *save all the transactions* to the database.
- It is responsible to make **permanent changes** in data.
- It ends the current transaction by saving database chnages & starts a new transaction.
- It is transactional command.
- When we use any DML command like INSERT, UPDATE or DELETE, the changes made by these commands are not permanent, until the current session is closed, the changes made by these commands can be rolled back.
- To avoid that, we use the COMMIT command to mark the changes as permanent.

TCL: Transaction Control Language

- **COMMIT:**
- **Syntax:**

- COMMIT; or COMMIT work;

- **Example:**
- **Consider table** student (Rollno, Name, Class, Marks, age)

```
DELETE FROM Students WHERE RollNo =25;

SELECT  * FROM Students WHERE RollNo =25;

COMMIT;
```

- **Or**

```
Select * FROM Students WHERE RollNo =25;
COMMIT;
```

TCL: Transaction Control Language

- **ROLLBACK:**

- It ends the current transaction by *removing* database changes & starts a new transaction.
- It is also ***used with SAVEPOINT command*** to jump to a savepoint in an ongoing transaction.

- Used to undo transaction that have not already been saved to the database.

Syntax:

```
ROLLBACK;
```

Or

```
ROLLBACK TO savepoint_name;
```

Example:

```
DELETE FROM Students WHERE RollNo =25;
ROLLBACK;
```

TCL: Transaction Control Language

SAVEPOINT

It is used to roll the transaction back to a certain point without rolling back the entire transaction.

Also defines as a breakpoints for the transaction to enable partial rollback.

It is treated as marker to devide lenghty transaction to a smaller one.

Syntax:

SAVEPOINT <SAVEPOINT_NAME >;

Used with rollback:

Rollback to <SAVEPOINT_NAME>;

TCL: Transaction Control Language

SAVEPOINT

Example:

Savepoint sp1;

DELETE FROM Students WHERE RollNo =1;

Savepoint sp2;

DELETE FROM Students WHERE RollNo =2;

Savepoint sp3;

DELETE FROM Students WHERE RollNo =3;

- *We want to undo the sp2*

Rollback to sp2;

TCL: Transaction Control Language

- COMMIT
- ROLLBACK
- SAVEPOINT

- BEGIN
- TRANSACTION
- UPDATE employees SET empname=’bob’ WHERE empid=’001’
- UPDATE employees SET empname =’bob’ WHERE city=’hyderabad’
- IF @@ROWCOUNT=5
- COMMIT
- TRANSACTION
- ELSE
- ROLLBACK
- TRANSACTION

View in SQL

- A view is a database object that has *no values*.
- Its contents are *based on the base table*.
- It contains *rows* and *columns* similar to the *real table* but does not contain any data of its own.
- In SQL, a view is a *virtual table* based on the *result-set of an SQL statement*.
- It is also created by a query by *joining one or more tables*.
- You can add *SQL statements* and *functions* to a view and present the data as if the data were coming from one single table.
- A view is created with the *CREATE VIEW* statement.
-

View in SQL

- MySQL allows us to create a view in mainly two ways:

✓ MySQL Command line client
✓ MySQL Workbench

- We can create a new view by using the ***CREATE VIEW*** and ***SELECT*** statement.

- SELECT statements are used *to take data from the source table to make a VIEW.*

- ***Syntax***

CREATE [OR REPLACE] **VIEW** view_name **AS**
SELECT columns
FROM tables
[**WHERE** conditions];

View in SQL

- ***Parameters:***

- The view syntax contains the following parameters:

- **OR REPLACE**: It is optional. It is used when a *VIEW already exists*.

- If you do not specify this clause and the VIEW already exists, the CREATE VIEW statement will return an error.

- **view_name**: It specifies the name of the VIEW that you want to create in MySQL.

- **WHERE conditions**: It is also optional.

- It specifies the conditions that must be met for the records to be included in the VIEW.

View in SQL

- **Example:**

- Let us understand it with the help of an example. Suppose our database has a table **student**, and we are going to create a view based on this table.

- Thus, the below example will create a VIEW name " **Stud_info**" that creates a virtual table made by taking data from the table **student**.

- **Syntax:**

```
CREATE VIEW  Stud_info  AS
SELECT Rollno, Sname
 FROM  student;
```

View in SQL

- **To see or display view:**

SELECT * **FROM** view_name;

- **Example:**

SELECT * **FROM** Stud_info;

- **Update VIEW:**

- the ***ALTER VIEW*** statement is used to modify or update the already created VIEW without dropping it.

- **Syntax:**

ALTER VIEW view_name **AS**
SELECT columns
FROM table
WHERE conditions;

View in SQL

- **Example:**

ALTER VIEW Stud_info **AS**
SELECT Rollno, Sname, Class
FROM Student;

- Once the execution of the **ALTER VIEW** statement becomes successful, MySQL will update a view and stores it in the database.

➢ **Drop VIEW:**

- We can drop the existing ***VIEW*** by using the **DROP VIEW** statement.

DROP VIEW [IF EXISTS] view_name;

View in SQL

- **Create View with JOIN Clause:**

- complex example of view creation that involves multiple tables and uses a **join** clause.

- Two tables:

Student

Roll _no	Sname	Class
1	Abhay	TE A
2	Riyansh	TE A
3	Eva	TE A

Stud_contact

Roll _no	Phone_no	Address
1	7778889907	Pune
2	8897656566	solapur
3	9876546789	mumbai

View in SQL

```
CREATE VIEW Stud_vw

AS

SELECT  s.sname, s.class, c.phone_no

FROM student s,  Stud_contact c

WHERE  S.Roll_no = c. Roll_no;
```

- We can display or see the view using the ***SELECT*** statement

SQL Functions

❖ SQL Functions:

- Aggregate Functions
- Scalar Functions/String Functions
- Date and Time Functions
- Conditional Functions
- Logical Functions
- Conversion Functions
- Window Functions

SQL Functions

➢ Scalar Functions / String Functions:

- Scalar functions are used to ***perform calculations on a single value*** and *return a single result*.
- Some examples of scalar functions in SQL include –

- **CONCAT():** Concatenates two or more strings.

- ***Example:*** we can use CONCAT to combine the **Name** and **city** columns in our "**Emp**" table:

```
Select concat(Name ," ", Name ) from Emp;
```

- **LEN()/ Length():** Returns the length of a string.

- **Syntax:**

  - SELECT **LEN**(column_name) FROM table_name;

- **Example:**

```
SELECT Name, LEN(City) as LengthOfCity  FROM Emp;
```

SQL Functions

- **Scalar Functions / String Functions:**

- **UPPER() / UCASE():** Converts a string to uppercase.
  - is used to convert value of string column to Uppercase characters.
- **Syntax:**

  SELECT **UCASE/Upper**(column_name) from **table-name**;
- **Example**: consider table Emp

  SELECT **UCASE**(name) FROM **emp;**
- **LOWER() / LCASE():**
  - is used to convert value of string column to Lower case characters.
- **Syntax:**

  SELECT **LCASE/Lower**(column_name) from **table-name**;
- **Example**: consider table Emp

  SELECT **LCASE/Lower** (name) FROM **emp**;

SQL Functions

- **Scalar Functions / String Functions:**

- **MID() function:**

- MID function is used to ***extract substrings from column values*** of string type in a table.

Syntax:

SELECT **MID**(column_name/string, start, length) from table-name;

Example:

1. Extract a substring from a string (start at position 5, extract 3 characters)

SELECT **MID**("Sinhgad", 5, 3) AS ExtractString;

OutPut: gad

2. SELECT **MID**(name,2, 2) FROM emp;

SQL Functions

➢ Scalar Functions / String Functions:

- **LEFT():** Extracts a specified number of characters from the beginning/left of a string/column.

Syntax: 1. Select LEFT(*string*, *number_of_chars*);

2. Select LEFT(*column_name*, *number_of_chars*) from <Table_name>;

String: Required. The string to extract from

number_of_chars: Required. The number of characters to extract.

- **Example:** 1. Extract 4 characters from the string in the "Riyansh" from left.
- SELECT LEFT('Riyansh', 4) AS ExtractString;
- **Output: Riya**

2. SELECT **LEFT**(Name, 5) AS ExtractString FROM student;

SQL Functions

➢ Scalar Functions / String Functions:

- **RIGHT():** Extracts a specified number of characters from the RIGHT side of a string/column.

Syntax: 1. Select **RIGHT**(*string*, *number_of_chars*);

2. Select RIGHT(*column_name*, *number_of_chars*) from <Table_name>;

String: Required. The string to extract from

number_of_chars: Required. The number of characters to extract.

- **Example:** 1. Extract 4 characters from the string in the “Riyansh" from left.
- SELECT **Right**('Riyansh', 4) AS ExtractString;
- **Output:** **ansh**

2. SELECT **Right**(Name, 5) AS ExtractString FROM student;

SQL Functions

- **LPAD:** Left-pads a string with another string, to a certain length.
- **Syntax**: 1. select **LPAD**(*string*, *length*, *lpad_string*);

  2. select **LPAD**(*Column_name*, *length*, *lpad_string*) from table_name;

Example: 1. Left-pad the string with “TE", to a total length of 9:

SELECT LPAD("Riyansh", 9, "TE“);

Output: **TERiyansh**

2. SELECT LPAD("Riyansh", 9, "TE“) as Name;

Output: **Name**

TERiyansh

3. select **LPAD**(*Name*, *9*, *”Sathe”*) from student;

Note: As per column values it will display output.

SQL Functions

- **RPAD:** Left-pads a string with another string, to a certain length.

- **Syntax**: 1. select **RPAD**(*string*, *length*, *lpad_string*);

  2. select **RPAD**(*Column_name*, *length*, *lpad_string*) from table_name;

Example: 1. Right pad the string with “TE", to a total length of 9:

SELECT **RPAD**("Riyansh", 9, "TE");

Output: **RiyanshTE**

2. SELECT **RPAD**("Riyansh", 9, “TE”) as Name;

Output: **Name**

RiyanshTE

3. select **RPAD**(*Name*, *9*, *”Sathe”*) from student;

Note: As per column values it will display output.

SQL Functions

- **REVERSE:** Reverses a string and returns the result.

- **Syntax:**

1. SELECT **REVERSE**(“String");

Or SELECT **REVERSE**(“String ") as reverse_name;

2. SELECT **REVERSE**(Column_Name) from Table_name;

- **Example: 1. display Name Riyansh in reverse**

SELECT **REVERSE**(“Riyansh");

- **Output:** **hsnayiR**

2. SELECT **REVERSE**(Name) from Student;

SQL Functions

Numeric Functions:

Round()

SUM()

MAX()

MIN()

Count()

Avg()

SQL Functions

- **Round:**
- is used to round a numeric field to number of nearest integer.
- It is used on Decimal point values.

- **Syntax:**
- **1.** SELECT **ROUND(***number*, *decimals***);**
- 2. SELECT ROUND(column_name, decimals) from table-name;

Number *:* Required.The number to be rounded

Decimals: Optional. The number of decimal places to round *number* to. If omitted, it returns the integer (no decimals)

- **Example:**

1. SELECT ROUND(135.375, 1);

OutPut: 135.3

2. SELECT ROUND(salary) from emp;

SQL Functions

- **SUM():**

- **Syntax:**

  SELECT SUM(Column_Name) FROM Table_name;

- **Example:**

  SELECT SUM(Marks) FROM Student;

SQL Functions

- **String Functions:**

- **LTRIM()** - Removes the leading whitespace of the string
- **RTRIM()** - Removes the trailing whitespace of the string
- **TRIM()** - Removes both leading and trailing whitespace of the string
- **SUBSTRING()** - Extracts a specific portion of a string
- **REPLACE()** - Replaces all occurrences of a specified string with another string

SQL Functions

- **String Functions:**
- **TRIM():** Remove leading and trailing spaces from a string
- **Syntax:**
  1. Select TRIM(String);
  2.
- **Example:**

  Select TRIM(‘ Riyansh ’);
- **OutPut:** Riyansh

- **LTRIM()** - Removes the leading whitespace of the string
- Example:

  Select TRIM(‘ Riyansh ’)

SQL Functions

- **String Functions:**
- **SUBSTRING()**
- Extract a substring from a string (start at position 5, extract 3 characters):

Syntax:

1. Select SUBSTRING(*string*, *start*, *length*);
2. Select SUBSTRING(*column_name*, *start*, *length*) from table_name;

String : Required. The string to extract from

Start: Required. The start position.

Length: Optional.

Example:1

SELECT SUBSTRING("Riyansh", 5, 3)

Output: nsh

2. Select SUBSTRING(*Name*, *5*, *3*) from Student;

SQL Functions

- **String Functions:**
- **Replace:**
- replaces all occurrences of a substring within a string, with a new substring
- **Syntax:**

1. Select REPLACE(*string*, *substring*, *new_string*);
2. Select REPLACE(Column_name, *substring*, *new_string*);

String: Required. The original string

Substring: Required. The substring to be replaced.

new_string: Required. The new replacement substring

Example:

SELECT **REPLACE**("Riyansh", "Riyansh", "Evanshika");

Output: Evanshika

SQL Functions

- **Date and Time Functions:**

- SQL also provides a number of functions for working with date and time values.

- **NOW()** - Returns the current date and time
- **CURRENT_DATE()** - Returns the current date
- **CURRENT_TIME()** - Returns the current time
- Current_Timestamp()-Return the current date and time
- **YEAR()** - Returns the year of a date
- **MONTH()** - Returns the month of a date
- **DAY()** - Returns the day of a date

SQL Functions

- **Date and Time Functions:**

- SQL also provides a number of functions for working with date and time values.

- **NOW()** - Returns the current date and time.
- The date and time is returned as "YYYY-MM-DD HH:MM:SS"
- **Syntax:**
- SELECT NOW() FROM table_name;

- **Example:**
- selects the product name, and price for today from the "Products" table:

- SELECT ProductName, Price, Now() AS PerDate FROM Products;

SQL Functions

- **Date and Time Functions:**

- SQL also provides a number of functions for working with date and time values.

- **CURRENT_DATE() -** Returns the current date

```
SELECT CURRENT_DATE();
```

OutPut: 2024-07-21

- **CURRENT_TIME() -** Returns the current time

```
SELECT CURRENT_TIME();
```

Output:

17:08:35

SQL Functions

- **Date and Time Functions:**

- SQL also provides a number of functions for working with date and time values.

- **Current_Timestamp()**-Return the current date and time

- SELECT CURRENT_TIMESTAMP();

- 2024-07-21 17:09:44

- **YEAR()** - Returns the year of a date

SELECT YEAR("2017-06-15");

2017

SQL Functions

- **Date and Time Functions:**

- **MONTH()** - Returns the month of a date

```
SELECT MONTH("2017-06-15");
```

- **Output: 6**

```
SELECT MONTH(CURDATE());
```

- **Output: 7**

- **DAY()** - Returns the day of a date.

```
SELECT DAY("2017-06-15");
```

Output: 15

```
SELECT DAY(CURDATE());
```

Output: 21

SQL Functions

- **Conditional Functions:**

- SQL also provides a number of functions that perform different actions based on certain conditions. Some examples of conditional functions in SQL include –
- **CASE -** evaluates a list of conditions and returns a result for the first condition that is met.

- **IF -** return a specified value if the condition is met, otherwise return another specified value.

- **COALESCE -** return the first non-null expression among multiple expressions.

SQL Functions

- **Conditional Functions:**

- **Example:**

```
SELECT order_id, total_cost,
CASE
WHEN total_cost > 100 THEN 'expensive'
WHEN total_cost > 50 THEN 'moderately priced'
ELSE 'inexpensive'
END as
"price range"
FROM orders;
```

SQL Functions

- **Logical Functions:**

- SQL provides a ***set of logical functions*** that return a *Boolean value*, which can be either true or false.

- Some examples of logical functions in SQL include –

- **AND -** Returns true if both the conditions are true
- **OR -** Returns true if at least one of the conditions is true
- **NOT -** Negates a boolean value

Example:

SELECT name, Class, div FROM student WHERE Class = ‘TE” AND div= “A”;

SQL Functions

- **Conversion Functions:**

- SQL provides a number of functions that can be used to convert data from one type to another.
- Some examples of conversion functions in SQL include –

- **CAST()** - Converts a value from one data type to another
- **CONVERT()** - Converts a value from one data type to another (This function is specific for some database vendors like SQL Server)
- **TO_DATE()** - Converts a string to a date value
- **TO_TIME()** - Converts a string to a time value
- **TO_TIMESTAMP()** - Converts a string to a timestamp value

SQL Functions

- **Conversion Functions:**

- SQL provides a number of functions that can be used to convert data from one type to another.
- Some examples of conversion functions in SQL include –

- **CAST()** - Converts a value from one data type to another
- **CONVERT()** - Converts a value from one data type to another (This function is specific for some database vendors like SQL Server)
- **TO_DATE()** - Converts a string to a date value
- **TO_TIME()** - Converts a string to a time value
- **TO_TIMESTAMP()** - Converts a string to a timestamp value

SQL Functions

- **Window Functions:**
- SQL provides a set of functions that can be ***used to perform calculations across a set of rows*** that are related to the ***current row***.

- These functions are known as window functions.

- Some examples of window functions in SQL include –

- **RANK()** - Assigns a unique rank to each row within a result set, based on the values in one or more columns.

- **DENSE_RANK()** - Assigns a unique rank to each row within a result set, based on the values in one or more columns, but does not leave gaps in the ranking sequence when there are ties.

- **ROW_NUMBER()** - Assigns a unique number to each row within a result set, based on the order specified in the ORDER BY clause of the function.

SQL JOINS

- JOIN means "***to combine two or more tables***".
- In SQL, JOIN clause is ***used to combine the records from two or more tables*** in a database.
- JOINS are used with ***SELECT*** statement.
- It is used to ***retrieve data from multiple tables***.

✓ ***Types of SQL JOIN:***

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN

SQL JOINS

Emp

EmpID	Emp_name	City	Salary
1	Abhay	solapur	200000
2	Riyansh	Pune	300000
3	Kritika	Mumbai	100000
4	Rahul	Latur	500000
5	Rajesh	Hydrabad	200000
6	Sahil	Pimpri	400000

Dept

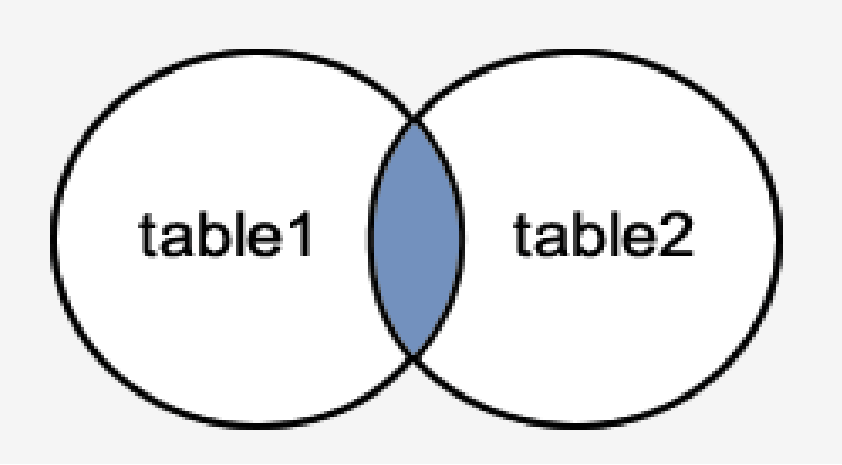
DeptID	EmpID	Department
101	1	Testing
102	2	Development
103	3	Designing
104	4	Development

SQL JOINS

1. INNER JOIN:

- In SQL, INNER JOIN selects records that have ***matching values in both tables*** as long as the *condition is satisfied*.

- It *returns the combination of all rows* from both the tables where the *condition satisfies.*



SQL JOINS

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example:

```
SELECT Emp.Emp_name,  Dept. Department
FROM Emp
INNER JOIN Dept
ON  Emp.EMP_ID =  Dept.EMP_ID;
```

SQL JOINS

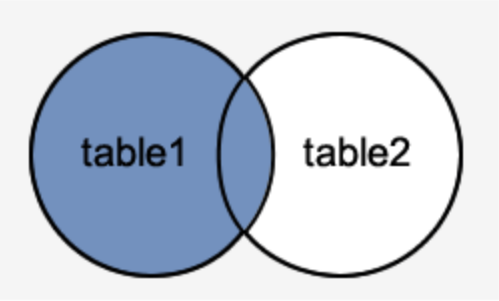
Emp_Name	Department
Abhay	Testing
Riyansh	Development
Kritika	Designing
Rahul	Development

SQL JOINS

2. LEFT JOIN/ Left Outer Join

- The SQL left join returns all the values from left table and the matching values from the right table.

- If there is no matching join value, it will return NULL.



Syntax:

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1
LEFT [Outer]  JOIN table2
ON table1.matching_column = table2.matching_column;
```

SQL JOINS

- **Example:**

SELECT Emp.Emp_name, Dept.Department
FROM **Emp**
LEFT [Outer] JOIN **Dept**
ON **Dept. EmpID = Emp.EmpID**;

Output

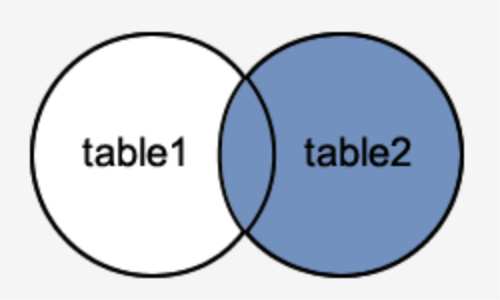
Emp_Name	Department
Abhay	**Testing**
Riyansh	**Development**
Kritika	**Designing**
Rahul	**Development**
Rajesh	**Null**
Sahil	**Null**

SQL JOINS

- **RIGHT [outer] JOIN**:

- In SQL, RIGHT JOIN returns all the values from the rows of right table and the matched values from the left table.

- If there is no matching in both tables, it will return NULL.



SQL JOINS

- **Syntax:**

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

- **Example:**

```
SELECT  Emp.EMP_NAME,  Dept.DEPARTMENT
FROM Emp
RIGHT JOIN Dept
ON Dept.EMP_ID = Emp.EMPID;
```

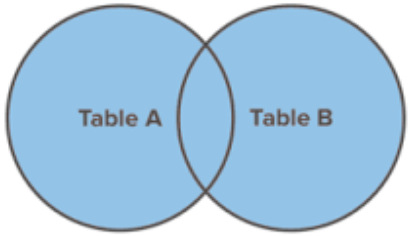
SQL JOINS

Emp_Name	Department
Abhay	Testing
Riyansh	Development
Kritika	Designing
Rahul	Development

SQL JOINS

4. FULL JOIN: FULL[OUTER] JOIN

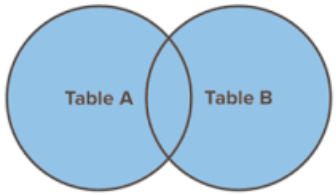
- In SQL, FULL JOIN is the result of a combination of both left and right outer join.
- Join tables have all the records from both tables.
- It puts NULL on the place of matches not found.



SQL FULL JOIN

Syntax:

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```



SQL FULL JOIN

SQL JOINS

- **Example:**

```
SELECT Emp.Emp_Name,  Dept.Department
FROM Emp
FULL [outer] JOIN Dept
ON  Dept.EMPID = Emp.EMPID;
```

Output

Emp_Name	Department
Abhay	Testing
Riyansh	Development
Kritika	Designing
Rahul	Development
Rajesh	Null
Sahil	Null

Aggregate function in SQL

- Performs a calculation on multiple values and returns a single value.
- Or
- used to perform the calculations on multiple rows of a single column of a table

- SQL provides many aggregate functions that include avg, count, sum, min, max, etc.

- An aggregate function *ignores NULL values* when it performs the calculation, *except for the count function*.

- They allow us to perform calculations on large data sets quickly and efficiently.

Aggregate function in SQL

- **Example**: these functions generate *statistical reports*, *perform financial analysis*, and *manage inventory levels*.

- we can easily *calculate the average price* of all products in our inventory or find the total sales for a particular time.

- **Syntax:**

- **Select Agg_fun(column_name) from <table_name> ;**

- An aggregate function in SQL returns one value after calculating multiple values of a column.

- We often use aggregate functions with the GROUP BY and HAVING clauses of the SELECT statement.

- Various types of SQL aggregate functions are:

- Count()
- Sum()
- Avg()
- Min()
- Max()

- **COUNT FUNCTION:**

- COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.

- COUNT(*) considers duplicate and Null.

- **Select count(*) from <table_name>;**

Example:

- **Select count(*) from student;**

- **Output:**

Student

Roll_no	Name	Marks	Class
1	Kalpesh	40	TE A
2	Swamini	50	TE A
3	Apurva	30	TE B
4	Atherva	60	TE B
5	Riyansh	20	TE A
6	Apurva	70	TE A

- **COUNT FUNCTION with where:**

Example:

- SELECT COUNT(*) FROM student WHERE Marks>=40;

- **Output: 4**

- **COUNT() with DISTINCT :**

Used to count the no. of values in a column without duplication.

Select **count(distinct column)** from <table_name>;

Select **count(distinct name)** from student;

output: 5

- **COUNT(column) :**

Used to count the no. of values in a column..

Select **count(column)** from <table_name>;

Select **count(name)** from student;

output: 6

COUNT() with GROUP BY:

```
SELECT class, COUNT(*) FROM student GROUP BY class;
```

Output: TE A= 4

TE B= 2

```
SELECT Class, COUNT(*) FROM student GROUP BY Class
  HAVING COUNT(*)>2;
```

Output:

TE A 4

- **SUM Function:**

- Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

- Select **sum(column_name)** from <table_name>;

- **Example:**

- Select **sum(Marks)** from student;

- **Output**: 270

Sum with where:

SELECT SUM(marks) FROM student WHERE marks > 50;

Output:

Sum with group by:

SELECT class,SUM(marks) FROM student WHERE marks>30 GROUP BY class;

Output:

- TE A 160
- TE B 60

- **Sum Function with Having:**

```
SELECT Class, SUM(Marks)
FROM student
GROUP BY Class
HAVING SUM(Marks)> 90;
```

Output: TE A 180

- **AVG function:**

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

```
SELECT AVG(marks)  FROM student;
```

- **MAX Function:**

- MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

```
SELECT MAX (marks)  FROM student;
```

- **MIN Function:**

- MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

```
SELECT Min (marks)  FROM student;
```

SQL Index

- An index is a data structure that allows us to add indexes in the existing table.
- It enables you to improve the *faster retrieval of records* on a database table.
- It creates an ***entry*** *for each value* of the indexed columns.
- We use it to ***quickly find the record without searching each row*** in a database table whenever the table is accessed.
- We can ***create an index*** by *using one or more* ***columns*** of the table for efficient access to the records.
- To create index use ***CREATE INDEX*** Statement.

SQL Index

- When a table is created with a ***primary key or unique key***, it automatically creates a special index named **PRIMARY**.
- We called this index as a ***clustered index***.
- All *indexes other than PRIMARY indexes* are known as a ***non-clustered index or secondary index.***
- ***Types of Index:***

1. ***Clustered index***
2. ***Non-clustered index or secondary index.***

SQL Index

- ***CREATE INDEX Statement:***
- **Simple and Unique Index**
- **Syntax:**

CREATE INDEX [index_name] **ON** [table_name] (**column** names) ;

- **index_name** is the name of the index,
- **table_name** is the name of the table to which the index belongs, and
- the **column_names** is the list of columns.

➢ ***Unique index*:** does not allow duplicate values.

CREATE UNIQUE INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);

SQL Index

- creates an index named "**idx_Lname**" on the "**LName**" column in the "**Student**" table:

CREATE INDEX idx_Lname ON Student(LName);

- to create an index on a combination of columns, you can list the column names within the parentheses, separated by commas:

CREATE INDEX idx_Fname ON Student(Fname, LName);

SQL Index

➢ **Displaying INDEX Information**:

- use the ***SHOW INDEX*** command to *list out all the indexes* associated with a table.

SHOW INDEX FROM *table_name*;

Example:

SHOW INDEXES FROM student;

SQL Index

- **Drop INDEX** : The *DROP INDEX* statement is used to delete an index in a table.

- **DROP INDEX** index_name **ON** table_name;

Example: **DROP INDEX idx_Lname ON** student;

OR

ALTER TABLE *table_name* DROP INDEX *index_name*;

Example:

ALTER TABLE student **DROP INDEX *idx_Lname*;**

SQL Index

- ***CREATE INDEX Statement at the time of table creation:***

- we create an index ***at the time of table creation*** in the database.

- The following statement creates a table with an index that contains two columns col2 and col3.

- Syntax:

```
CREATE TABLE t_index(
                        col1 INT PRIMARY KEY,
                        col2 INT NOT NULL,
                        col3 INT NOT NULL,
                         col4 VARCHAR(20),
            INDEX (col2,col3)
);
```

SQL SEQUENCE

- is an ***arrangement of integers generated in the ascending order*** (1, 2, 3, and so on) on specific demand.
- Sequences are used in the databases to ***generate unique numbers***.
- Many applications require each row of a table to contain a distinct value, such as student roll number in student_table, employee numbers in HR, customer ID in CRM, etc.
- To fulfill this type of arrangement, we use sequences that provide an easy ***way to generate them.***
- MySQL ***does not provide any built-in function to create a sequence*** for a table's rows or columns.

SQL SEQUENCE

- Create Sequence Using ***AUTO_INCREMENT.***

- The simplest way for creating a sequence in MySQL is by defining the column as ***AUTO_INCREMENT*** during table creation, which should be a ***primary key*** column.

- The following are the ***rules***:

  - We can create only ***one AUTO_INCREMENT column*** in each table, and

  - the ***data type of this column*** must be an ***integer.***

SQL SEQUENCE

- The ***AUTO_INCREMENT*** column should also have either ***PRIMARY*** or ***UNIQUE KEY*** ***indexing***.

- The ***AUTO_INCREMENT*** column must contain ***NOT NULL***

- *It* ***automatically adds*** the ***NOT NULL*** constraint to the column ***implicitly*** when we set the column as an ***AUTO_INCREMENT*** attribute.

SQL SEQUENCE

- First, create a ***new table*** and make sure that there is one column with the ***AUTO_INCREMENT*** attribute and that too, as ***PRIMARY KEY***.
- **Syntax:**

```
CREATE TABLE  <table_name>  (
        Col1 data type(size)  NOT NULL  AUTO_INCREMENT,
            PRIMARY KEY (col1),
        Col2 VARCHAR(30) NOT NULL
                                    );
```

- **Example:**

```
CREATE TABLE  Student (
        roll_no INT  NOT NULL AUTO_INCREMENT,
        PRIMARY KEY (roll_no),
       FName VARCHAR(30) NOT NULL,
       LName VARCHAR(30) NOT NULL);
```

SQL Operators

- An operator is a *reserved word or a character* used primarily in an SQL statement's WHERE clause to perform operation(s),

- such as comparisons and arithmetic operations.

- These Operators are used to *specify conditions in an SQL statement* and to *serve as conjunctions for multiple conditions* in a statement.

.

- ✓ **Arithmetic Operators**
- ✓ **Comparison operators**
- ✓ **Logical Operators**
- ✓ **Range operators**
- ✓ **Pattern matching operators**
- ✓ **Set operators**

SQL Operators: Set Operation

The SQL Set operation is used to ***combine the two or more SQL SELECT statements:***

- Types of Set Operation:
  - Union
  - UnionAll
  - Intersect
  - Minus

SQL Operators: Set Operation

1. Union:

- The SQL Union operation is ***used to combine the result of two or more SQL SELECT queries.***
- all the ***number of columns and datatypes*** must be ***same in both*** the tables on which UNION operation is being applied.
- The union operation *eliminates the duplicate rows* from its result set.
- or
- Used to display ***all distinct rows*** selected by two queries.
- ***Combine the output of two or more queries*** into a ***single set of rows & columns.***

SQL Operators: Set Operation

- **Syntax:**

```
SELECT column_name FROM table1
UNION
SELECT column_name FROM table2;
```

- **Example:**

```
SELECT * FROM table1
UNION
SELECT * FROM table2;
```

SQL Operators: Set Operation

Table1

ID	Name
1	Rishabh
2	Bhakti
3	Bhupal

.

Table2

ID	Name
3	Bhupal
4	Jay
5	Pratik

ID	Name
1	Rishabh
2	Bhakti
3	Bhupal
4	Jay
5	Pratik

SQL Operators: Set Operation

- **Intersect:**
- It is used to combine two SELECT statements.
- returns the ***common rows*** from both the SELECT statements.
- the number of datatype and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.

SQL Operators: Set Operation

- **Intersect:**

```
SELECT column_name FROM table1
        INTERSECT
SELECT column_name FROM table2;
```

- **Example:**

```
SELECT * FROM First
            INTERSECT
SELECT * FROM Second;
```

SQL Operators: Set Operation

- **Intersect: OutPut**

ID	Name
3	Bhupal

SQL Operators: Set Operation

4. Minus:

- It combines the result of two SELECT statements.
- Minus operator is used to ***display the rows which are present in the first query*** but *absent in the second query*.
- It has no duplicates and data arranged in ascending order by default.

```
SELECT column_name FROM table1
MINUS
SELECT column_name FROM table2;
```

SQL Operators: Set Operation

4. Minus:

Output:

ID	Name
1	Rishabh
2	Bhakti

SQL Operators

➢ Pattern matching operators:

- Like and not like operator
- Is null and Is not null operator

➢ Like and not like operator:

- The LIKE operator in SQL pattern matching is used for standard pattern matching.

- By using LIKE operator, we can compare one value exactly to one other value.
- There are two wild card characters:

- Percent sign: (**%**) **matches all string**
- Underscore **:** **(_)** **matches any single character**

SQL Operators

➢ Pattern matching operators:

- Like and not like operator:

- **Using LIKE Clause With %**
- In SQL pattern matching, the **%** (percentage) sign is used to match an arbitrary number of characters.

✓ It can be used in several ways:

- **a%** : selects all entries that start with the letter ***a***
- **%b** : selects all entries that end with the letter ***b***
- **a%b** : selects all entries that start with ***a*** and end with ***b***.
- **abc%:** selects all entries that start with ***abc***.

SQL Operators

- **Pattern matching operators:**

- **Like and not like operator:**

- Select <columns>/ * from table where <column> LIKE pattern

- **Example:**

- Select all entries where the fisrt name starts with ***B***

SELECT * FROM **student** WHERE Fname LIKE 'b%';

- Select all entries where the student's **Last name** starts with **ab%;**

SELECT * FROM **student** WHERE **Fname** LIKE **' ab%';**

SQL Operators

- **Pattern matching operators:**

- **Like and not like operator:**

- Select all entries where the student's First name has ***bh*** in it

SELECT * FROM **student** WHERE **Fname LIKE** **'% *bh* %';**

- **LIKE clause with _ :**
- is used to match exactly one arbitrary character. It can also be used in a number of ways.

- **Example:** Select all entries where the student's ***First name has exactly five letters.***

- **Example:**

SELECT * FROM **student** WHERE **Fname** LIKE '____';

SQL Operators

- **Pattern matching operators:**
- **Like and not like operator:**

Select all entries where the students first name starts with ***Abha_*** and has exactly five letters in them

SELECT * FROM student WHERE Fname LIKE ' *Abha* _';

Using LIKE Clause with % And _ Operator in A Single Query:

We can also combine and use both % and _ operators in a single query to pattern match

Example: Select all entries where the students First name has ***n*** as its second last letter

SELECT * FROM **student** WHERE **Fname** LIKE **'%n_';**

SQL Operators

- **Pattern matching operators:**

- **Like and not like operator:**

- WHERE **Fname LIKE '_r% '** :Finds any values that have "r" in the second position

- WHERE **Fname LIKE 'a_% ' :** Finds any values that start with "a" and are at least 2 characters in length

- WHERE **Fname LIKE 'a__%':** Finds any values that start with "a" and are at least 3 characters in length

- WHERE **Fname LIKE 'a%o'**: Finds any values that start with "a" and ends with "o"

SQL Operators

- **Pattern matching operators:**

- **Is null and Is not null operator**

- **IS NULL Operator:**

- This operator returns **TRUE** if the operand value is ***NULL(empty***) otherwise returns **FALSE**

SELECT * from student WHERE **age IS NULL**;

- **IS Not NULL Operator:**

- This operator returns **TRUE** if the operand value is Not NULL otherwise returns **FALSE**

SELECT * from student WHERE **age IS Not NULL**;

SQL Operators

- **Using LIKE Clause with NOT Operator:**
- we may also select those entries that do not follow a given pattern.
- we can exclude all those entries that match the specified pattern.
- Select all entries where the **first name** does not have ***l*** as its second letter

SELECT * FROM student WHERE name NOT LIKE '_l%';

SQL Operators

❑ Range operators:

- **BETWEEN Operator &**
- **NOT BETWEEN Operator**
- **IN operator**
- **NOT IN operator**

SQL Operators

❑ Range operators:

➢ Between Operator & not between Operator:

- The BETWEEN Operator in SQL is used to ***get the values within a specified range.***
- is used ***to compare the value in a column*****.**
- This operator ***tests a range of values with a column value*****.**
- is written as the word "***BETWEEN***" ***followed by a range of values.***
- ***begin*** and ***end values*** are *included*.
- BETWEEN operator is used in the WHERE clause with the following four SQL statements:
- SELECT Statement, UPDATE Statement, DELETE Statement, INSERT Statement

SQL Operators

- **Range operators:**

Between Operator & not between Operator

Student

Roll_no	Name	Marks	Class
1	Kalpesh	40	TE A
2	Swamini	50	BE A
3	Apurva	30	TE B
4	Atherva	60	TE B
5	Riyansh	20	BE B
6	Apurva	70	TE A

SQL Operators

- **Range operators:**

Between Operator :

Syntax:

SELECT ***column_names***

FROM ***table_name***

WHERE ***column_name*** BETWEEN *value1* AND *value2;*

Example:

SELECT * FROM

WHERE ***marks*** **BETWEEN** 30 **AND** *50;*

Output: 30, 40, 50

SQL Operators

- **NOT BETWEEN Operator:**

To display the products outside the range.

Syntax:

SELECT ***column_names***
FROM ***table_name***
WHERE ***column_name NOT BETWEEN*** *value1* ***AND*** *value2;*

Example:

SELECT * FROM
WHERE ***marks NOT BETWEEN*** 30 **AND** *50;*

Output: 20, 60, 70

SQL Operators

➢ IN operator:

- SQL IN is an operator used in a SQL query to help ***reduce the need to use multiple SQL "OR" conditions.***
- It is used in SELECT, INSERT, UPDATE or DELETE statement.
- he IN operator allows you to specify multiple values in a WHERE clause.
- **Syntax:**

SELECT *column_name(s)*

FROM *table_name*

WHERE *column_name* IN (*value1*, *value2*, ...);

SQL Operators

- **IN operator:**

Or

SELECT *column_name(s)*

FROM *table_name*

WHERE *column_name* **IN** (*SELECT STATEMENT*);

- If you want to fetch all the students from the student table whose class is either **TE A or TE B,** then you need to write the SQL Query using IN Operator as shown below.

- **SELECT * FROM** student **WHERE Class IN ('TE A', 'TE B');**

SQL Operators

- **IN operator:**

- **SELECT * FROM** student **WHERE Class IN ('TE A', 'TE B');**

- **Output: student**

Roll_no	Name	Marks	Class
1	Kalpesh	40	TE A
3	Apurva	30	TE B
4	Atherva	60	TE B
6	Apurva	70	TE A

SQL Operators

- **Arithmetic Operators:**

- It perform the ***mathematical operation on the numerical data*** of the SQL tables.

- These operators perform addition, subtraction, multiplication, and division operations on the numerical operands.

.

- **Following are the various arithmetic operators performed on the SQL data:**

- SQL Addition Operator (+)
- SQL Subtraction Operator (-)
- SQL Multiplication Operator (*)
- SQL Division Operator (/)
- SQL Modulus Operator (%)

SQL Operators

- **Arithmetic Operators:**

- **SQL Addition Operator (+):**

- performs the addition on the numerical data of the database table.

- In SQL, we can easily add the numerical values of two columns of the same table by specifying both the column names as the first and second operand.

- We can also add the numbers to the existing numbers of the specific column.

- **Syntax:**
- SELECT operand1 + operand2 from <table_name>;

SQL Operators

- **Arithmetic Operators:**
- **SQL Addition Operator (+):**
- **Syntax:**

SELECT **operand1** + **operand2** from <table_name>;

- Table : **Employee**

EmpID	Name	Salary	Emp Monthlybonus
1	Kalpesh	40000	300
2	Apurva	30000	200
3	Atherva	60000	400
4	Apurva	20000	200

SQL Operators

- **Arithmetic Operators:**

- **Example: 1**

- Suppose, we want to add **20,000** to the salary of each employee specified in the table. Then, we have to write the following query in the SQL:

SELECT ***Emp_Salary + 20000*** as ***Emp_New_Salary*** FROM Employee;

.

Example: 2

- Suppose, we want to add the Salary and monthly bonus columns of the above table, then we have to write the following query in SQL:

SELECT **Emp_Salary** + **Emp_Monthlybonus** **as** ***Emp_Total_Salary*** FROM Employee;

SQL Operators

- **Arithmetic Operators:**
- **SQL Subtraction Operator (-):**

- performs the subtraction on the numerical data of the database table.
- easily subtract the numerical values of two columns of the same table by specifying both the column names as the first and second operand.
- We can also subtract the number from the existing number of the specific table column.

✓ **Syntax:**

SELECT operand1 - operand2 from <table_name>;

SQL Operators

- **Arithmetic Operators:**

- **SQL Subtraction Operator (-):**

Example:

- Suppose we want to subtract 2,000 from the salary of each employee given in the **Employee_details** table. Then, we have to write the following query in the SQL:

```
SELECT Emp_Salary - 5000 as Emp_New_Salary FROM Employee;
```

SQL Operators

- **Arithmetic Operators:**

- **SQL Multiplication Operator (*):**

- The Multiplication Operator in SQL performs the Multiplication on the numerical data of the database table.

- In SQL, we can easily multiply the numerical values of two columns of the same table by specifying both the column names as the first and second operand.

Syntax:

SELECT operand1 * operand2 from <table_name>;

SQL Operators

- **Arithmetic Operators:**
- **SQL Multiplication Operator (*):**

Example: 1

- Suppose, we want to double the salary of each employee given in the **Employee** table.

- Then, we have to write the following query in the SQL:

SELECT ***Emp_Salary * 2*** as *Emp_New_Salary* FROM Employee;

- If we want to multiply **the Emp_Id** column to **Emp_Salary** column of that employee whose **Emp_Id** is **202**

Example: 2

SELECT Emp_Id * Emp_Salary FROM Employee WHERE Emp_Id = 202;

SQL Operators

- **Arithmetic Operators:**

SQL Division Operator (/):

- The Division Operator in SQL divides the operand on the left side by the operand on the right side.

- we can also divide the numerical values of one column by another column of the same table by specifying both column names as the first and second operand.

- We can also perform the division operation on the stored numbers in the column of the SQL table.

Syntax:

SELECT operand1 / operand2 from <Table_name>;

SQL Operators

- **Arithmetic Operators:**

SQL Division Operator (/):

Example: 1

- Suppose, we want to half the salary of each employee given in the Employee table.

```
SELECT  Salary / 2 as Emp_New_Salary FROM Employee;
```

Example: 2

```
SELECT  Salary / 2 as Emp_New_Salary FROM Employee where EmpId="2";
```

SQL Operators

- **Arithmetic Operators:**
- **SQL Modulus Operator (%):**
- provides the remainder when the operand on the left side is divided by the operand on the right side.

Syntax:

SELECT operand1 % operand2;

Example:

- If we want to get the remainder by dividing the numbers of First_operand column by the numbers of Second_operand column, then we have to write the following query in SQL:

SELECT ***First_operand % Second_operand*** as Remainder FROM Employee;

SQL Operators

- **Arithmetic Operators:**
- **SQL Modulus Operator (%):**

Number	First _operand	Second_ operand
1	56	4
2	32	8
3	89	9
4	18	10

SQL Operators

Comparison operators:

- SQL Equal Operator (=)
- SQL Not Equal Operator (!=)
- SQL Greater Than Operator (>)
- SQL Greater Than Equals to Operator (>=)
- SQL Less Than Operator (<)
- SQL Less Than Equals to Operator (<=)

SQL Operators

Comparison operators:

- **SQL Equal Operator (=)**

```
SELECT * FROM Employee_details WHERE Emp_Salary = 30000;
```

- **SQL Not Equal Operator (!=)**

```
SELECT * FROM Employee_details WHERE Emp_Salary != 20000;
```

- **SQL Greater Than Operator (>)**

```
SELECT * FROM Employee_details WHERE Emp_Id > 2;
```

- **SQL Greater Than Equals to Operator (>=)**

```
SELECT * FROM Employee_details WHERE Emp_Id >= 2;
```

SQL Operators

Comparison operators:

- **SQL Less Than Operator (<)**

```
SELECT * FROM Employee WHERE EmpId <2;
```

- **SQL Less Than Equals to Operator (<=)**

```
SELECT * FROM Employee WHERE EmpId <= 2;
```

Pl/SQL

- PL/SQL is a *block structured language.*
- The programs of PL/SQL are *logical blocks* that can contain *any number of nested sub-blocks.*
- PL/SQL stands for "*Procedural Language extension of SQL*" that is used in Oracle.
- PL/SQL is integrated with *Oracle database* (since version 7).
- The functionalities of PL/SQL usually extended after each release of Oracle database.
- Although PL/SQL is closely integrated with SQL language, yet it adds some programming constraints that are not available in SQL.

Pl/SQL

- **Message declaration**

- dbms_output.put_line (message/);

.

Pl/SQL

- **PL/SQL Variables:**

- A variable is a meaningful name which facilitates a programmer to store data temporarily during the execution of code.

- It helps you to manipulate data in PL/SQL programs.

- It is nothing except a name given to a storage area.

- Each variable in the PL/SQL has a specific data type which defines the size and layout of the variable's memory.

- A variable should not exceed 30 characters.

- Its letter optionally followed by more letters, dollar signs, numerals, underscore etc.

Pl/SQL

- **Naming rules for PL/SQL variables:**

✓ The *variable* in PL/SQL must follow some *naming rules* like other programming languages.

✓ The *variable_name* should not exceed 30 characters.

✓ Variable name should not be the same as the *table table's column* of that block.

.

✓ The name of the variable must begin with ASCII letter.

✓ The PL/SQL is *not case sensitive* so it could be either lowercase or uppercase.

✓ For example: v_data and V_DATA refer to the same variables.

✓ You should make your variable easy to read and understand, after the first character, it may be any number, underscore (_) or dollar sign ($).

✓ NOT NULL is an optional specification on the variable.

Pl/SQL

Declaration of variable:

Syntax:

Variable_name data tyape := value;

Example:

- a **integer** := 30;
- b char :=

Pl/SQL

```
DECLARE
        a integer := 30;
        b integer := 40;
        c integer;
         f real;
BEGIN
         c := a + b;
                  dbms_output.put_line('Value of c: ' || c);
         f := 100.0/3.0;
                  dbms_output.put_line('Value of f: ' || f);
END;
```

Pl/SQL

- **Variable Scope in PL/SQL:**

- PL/SQL allows nesting of blocks. A program block can contain another inner block.

- If you declare a variable within an inner block, it is not accessible to an outer block.

.

- There are two types of variable scope:

- ***Local Variable***: Local variables are the inner block variables which are not accessible to outer blocks.

- ***Global Variable***: Global variables are declared in outermost block.

Pl/SQL

```
DECLARE
 -- Global variables
  num1 number := 95;
  num2 number := 85;
BEGIN
  dbms_output.put_line('Outer Variable num1: ' || num1);
  dbms_output.put_line('Outer Variable num2: ' || num2);
  DECLARE
    -- Local variables
    num1 number := 195;
    num2 number := 185;
  BEGIN
    dbms_output.put_line('Inner Variable num1: ' || num1);
    dbms_output.put_line('Inner Variable num2: ' || num2);
  END;
END;
/
```

Pl/SQL: Block Structure

Syntax:

```
DECLARE
        <declarations section>
BEGIN
        <executable command(s)>
EXCEPTION
        <exception handling>
END;
```

Example:

```
DECLARE
        message varchar2(20):= 'Hello, World!';
BEGIN
        dbms_output.put_line(message);
END; /
```

Pl/SQL: Block Structure

- **Declarations:**
- This section starts with the keyword **DECLARE.**
- It is an optional section.
- defines all variables, cursors, subprograms, and other elements to be used in the program.

- **Executable Commands:**
- This section is enclosed between the keywords
- **BEGIN** and **END**
- it is a mandatory section.
- It consists of the *executable PL/SQL statements of the program.*
- It should have *at least one executable line of code*, which may be just a **NULL command** to indicate that nothing should be executed.

Pl/SQL: Block Structure

- ***Exception Handling:***

- This section starts with the keyword EXCEPTION.

- This optional section contains exception(s) that handle errors in the program.

- ***End***: line signals the end of the PL/SQL block.

Pl/SQL

- **Variable Attributes:**
- When you *declare a **PL/SQL variable** to hold the column values*,
- it must be of *correct data types and precision*, otherwise error will occur on execution.
- Rather than *hard coding* the data type and precision of a variable.
- PL/SQL provides the facility to declare a variable without having to specify a particular data type *using **%TYPE** and **%ROWTYPE** attributes*.
- These two attributes allow us to specify a *variable* and have that *variable data type* be defined by a table/view column or a PL/SQL package variable.
- A **%** sign serves as the attribute indicator

Pl/SQL

Data Types in PL/SQL:

Same as that of SQL but some advanced data types are there:

- Char(Size)
- Varchar (size)
- Date
- Number(p, s)

Range: p= 1 to 38 s= -84 to 127

- Long
- Raw
- Long Raw

Advanced Data types:

BLOB
CLOB
Bfile
%Types
%Rowtype
%RowId

Pl/SQL

Data Types in PL/SQL:

Advanced Data types:

BLOB:

- ✓ Used to store binary characters.
- ✓ Example: songs, video, image etc.
- ✓ Its storage space is 4GB

CLOB:

- ✓ Used to store Character variable.
- ✓ Its storage space is 4GB

Bfile:

- ✓ Used store value in pointer variable.
- ✓ Its storage space is 4GB

Pl/SQL

- **%TYPE:**
- It stores value of that ***variable whose data type is unknown*** and when we want the ***variable to inherit the data type of the table column.***

- Also, its value is generally being retrieved from an existing table in the database, hence it takes the data type of the column for which it is used.

.

- **Syntax:**
- Variable_name table_name.Column_name%TYPE;

- **Example:**

SALARY EMP.SAL % TYPE;

Pl/SQL

- **%ROWTYPE:**
- The %ROWTYPE attribute is used to ***declare a record type*** that represents a ***row in a table.***

- The record can store an entire row or some specific data selected from the table.

- A column in a row and corresponding fields in a record have the same name and data types.

- **Syntax**:

Variable_Name Table_name%Rowtype;

- **Example**:

SALARY EMP %rowTYPE;

Pl/SQL

```
DECLARE
EMPLOYEE EMP. % ROW TYPE;
BEGIN
EMPLOYEE.EMPNO := 2092;
EMPLOYEE.ENAME := 'Sanju';
Insert into EMP where (EMPNO, ENAME) Values (employee.empno, employee.
    ename);
dbms_output.put_line('Row Inserted');
END;
```

Pl/SQL

- **Control Statements:**

- PL/SQL IF
- PL/SQL Case
- PL/SQL Loop
- PL/SQL Exit Loop
- PL/SQL While Loop
- PL/SQL For Loop
- PL/SQL Continue
- PL/SQL GOTO

Pl/SQL

- **Control Statements:**

- **PL/SQL IF :**
- **Syntax: (IF-THEN statement):**

IF condition
THEN
Statement: {It **is** executed **when** condition **is true**}

- **END** IF;
- **Syntax: (IF-THEN-ELSE statement):**

IF condition
THEN
{...statements **to execute when** condition **is TRUE...**}
ELSE
{...statements **to execute when** condition **is FALSE...**}
END IF;

Pl/SQL

- **Syntax: (IF-THEN-ELSIF statement):**

IF condition1

THEN

{...statements **to execute when** condition1 **is TRUE...**}

ELSIF condition2

THEN

{...statements **to execute when** condition2 **is TRUE...**}

END IF;

Pl/SQL

- **Syntax: (IF-THEN-ELSIF-ELSE statement):**

IF condition1

THEN

{...statements **to execute when** condition1 **is TRUE...**}

ELSIF condition2

THEN

{...statements **to execute when** condition2 **is TRUE...**}

ELSE

{...statements **to execute when** both condition1 and condition2 are **FALSE**

...}

END IF;

Pl/SQL

- **Example:**

```
DECLARE
        a number(3) := 500;
BEGIN
        -- check the boolean condition using if statement
    IF( a < 20 ) THEN
        -- if condition is true then print the following
        dbms_output.put_line('a is less than 20 ' );
    ELSE
        dbms_output.put_line('a is not less than 20 ' );
    END IF;
        dbms_output.put_line('value of a is : ' || a);
    END;
```

Pl/SQL

- **PL/SQL Case Statement**

- The PL/SQL CASE statement facilitates you *to execute a sequence of statements based on a selector.*

- A selector can be anything such as variable, function or an expression.

- The CASE statement works like the **IF statement**, only using the keyword **WHEN**.

- A CASE statement is evaluated from top to bottom.

- If it get the condition TRUE, then the corresponding THEN calause is executed and the execution goes to the END CASE clause.

Pl/SQL

- **Syntax for the CASE Statement:** :

```
CASE [ expression ]
WHEN condition_1 THEN result_1 ;
  WHEN condition_2 THEN result_2;
  ...
  WHEN condition_n THEN result_n ;
ELSE result ;
END  CASE;
```

Pl/SQL

- **Example:**

```
DECLARE
  grade char(1) := 'A';
BEGIN
  CASE grade
    when 'A' then dbms_output.put_line('Excellent');
    when 'B' then dbms_output.put_line('Very good');
    when 'C' then dbms_output.put_line('Good');
    when 'D' then dbms_output.put_line('Average');
    when 'F' then dbms_output.put_line('Passed with Grace');
    else dbms_output.put_line('Failed');
  END CASE;
END;
```

Pl/SQL

- **PL/SQL Loop**
- The PL/SQL loops are used to repeat the execution of one or more statements for specified number of times.

- These are also known as iterative control statements.
- **Syntax:**

```
LOOP
    .
    Sequence of statements;

END LOOP;
```

❖ **Types of PL/SQL Loops**

- There are 4 types of PL/SQL Loops.
    - Basic Loop / Exit Loop
    - While Loop
    - For Loop
    - Cursor For Loop

Pl/SQL

- **PL/SQL While Loop:**
- PL/SQL while loop is used when a set of statements has to be executed as long as a condition is true.
- The condition is decided at the beginning of each iteration and continues until the condition becomes false.
- **Syntax:**

```
WHILE <condition>
 LOOP statements;
END LOOP;
```

- **Example:**

```
DECLARE
i INTEGER := 1;
BEGIN
WHILE i <= 10 LOOP
DBMS_OUTPUT.PUT_LINE(i);
i := i+1;
END LOOP;
END;
```

Pl/SQL

- **PL/SQL FOR Loop:**
- PL/SQL for loop is used when you want to execute a set of statements for a predetermined number of times.
- The loop is iterated between the start and end integer values.
- The counter is always incremented by 1 and once the counter reaches the value of end integer, the loop ends.

- **Syntax**:

```
FOR counter IN initial_value .. final_value LOOP
  LOOP statements;
END LOOP;
```

- initial_value : Start integer value
- final_value : End integer value

Pl/SQL

Example:

```
BEGIN
FOR k IN 1..10 LOOP
-- note that k was not declared
DBMS_OUTPUT.PUT._LINE(k);
END LOOP;
END;
```

Pl/SQL : Stored Procedure

- A stored procedure is a ***prepared or precompiled SQL code*** that you can save, so the code can be ***reuse*** over and over again.

- ***Named pl/sql block of code*** that have been ***compiled*** and ***performs*** a one or more tasks

- It is ***logically grouped set of SQL & PL/SQL statements.***

- It consist of ***header*** and ***body*** .

- **Header** consists of the **name of procedure** & ***parameters*** or ***variables*** passed to procedure.

- **Body** consist of :
  - ✓ Declaration section
  - ✓ Execution Section
  - ✓ Exception Section

Pl/SQL : Stored Procedure

- It can be used to ***perform a wide range of database oper***ations such as inserting, updating, or deleting data, generating reports, and performing complex calculations.

- Stored procedures are very useful because they allow you to ***encapsulate (bundle) a set of SQL statements as a single unit*** and ***execute*** them repeatedly with different parameters, making it easy to manage and reuse code.

- ***Two types of procedures in sql server:***

- SQL Server categorizes the stored procedures mainly in two types:

  ✓ User-defined Stored Procedures
  ✓ System Stored Procedures

Pl/SQL : Stored Procedure

- **Types of parameters:**

- **IN:** It is a default parameter. It passes the value to the subprogram.
- Referenced by procedure. The value of the parameter cannot be overwritten by procedure.

- **OUT:** It must be specified. It returns a value to the caller.
- Parameters can not be Referenced by procedure, but the value of the parameter can be overwritten by the procedure.

- **IN OUT:** It must be specified. It passes an initial value to the subprogram and returns an updated value to the caller.
- Can be referenced by procedure and value of the parameter can be overwritten by the procedure.

Pl/SQL : Stored Procedure

Syntax: in oracle

```
CREATE PROCEDURE procedure_name

        [List of Parameters]

AS /IS
              --Variable declaration
BEGIN

      -- SQL statements to be executed/executable statements

Exception

       --Exception section

END
```

Pl/SQL : Stored Procedure

Syntax:

```
CREATE PROCEDURE procedure_name
@parameter1 datatype,
 @parameter2 datatype
    AS /IS
            --Variable declaration
BEGIN

     -- SQL statements to be executed/executable statements

Exception

 --Exception section

END
```

Pl/SQL : Stored Procedure

- **Simple Example without parameter:**

```
CREATE PROCEDURE student_List
AS
BEGIN
   SELECT name, age, salary
   FROM STUDENT
   ORDER BY salary;
END;
```

- **Execution of procedure:** will display name ,age and salary columns with ascending order

```
EXEC student_List;
```

Or,

```
EXECUTE  student_List;
```

Pl/SQL : Stored Procedure

Example: with In parameter

- Create the stored procedure with the name '**GetStudentInfo**'.
- We then provide it with a single input parameter called *@StudentAge*.
- The stored procedure then selects all records from the '**Student**' table where the **StudentAge** matches the **input parameter**.

```
CREATE PROCEDURE GetStudentInfo
  @StudentAge INT
        AS
  BEGIN
        SELECT * FROM Student WHERE Age = @StudentAge
 END
```

Pl/SQL : Stored Procedure

Execution:

- Test it by executing it with different input parameters as shown in the query below :

EXEC **GetStudentInfo** @ StudentAge = 21

- This will return all columns from the **Student** table where the **Student**'s age is 21.

Pl/SQL : Stored Procedure

- **Procedure with OUT parameter:**

- The OUT parameter sends output value to the program.

- It allows us to return a value or set of values to the calling program.

- when using an OUT parameter, we must specify the keyword "OUT" after the parameter name when passing it to the stored procedure.

- create a stored procedure that takes the customer's ID as an input parameter and returns that customer's salary using an output parameter "@Salary" which holds the salary value returned by the stored procedure.

- The procedure body performs a SELECT statement to get the "Salary" column from the "CUSTOMERS" table, where the "ID" matches the input parameter.

Pl/SQL : Stored Procedure

- We use the the assignment operator "=" to assign the returned value to our output parameter "@Salary".

- **Procedure with OUT parameter:**

```
CREATE PROCEDURE GetCustomerSalary

@CustomerID INT,

@Salary DECIMAL(18,2) OUT

AS

BEGIN

SELECT @Salary = SALARY FROM CUSTOMERS WHERE ID = @CustomerID

END
```

Pl/SQL : Stored Procedure

- **Execution:**

- we first declare a variable ***"@CustSalary"*** of the same datatype as the output parameter ***"@Salary".***

- We then execute the stored procedure using the EXEC command, passing in the ***"@CustomerID*** " parameter and the ***"@Salary***" parameter as an ***output parameter***.

- Finally, we SELECT the value of the ***"@CustSalary***" variable to *display the customer's salary in the result set*.

```
DECLARE @CustSalary DECIMAL(18, 2)

EXEC GetCustomerSalary @CustomerID = 4, @Salary = @CustSalary OUT

SELECT @CustSalary AS 'Customer Salary'
```

Pl/SQL : Stored Procedure

- **Rename a Stored Procedure:**
- To rename a stored procedure in SQL, we use the built-in system stored procedure called sp_rename.

- **Syntax:**

```
sp_rename 'old_procedure_name', 'new_procedure_name';
```

- **Example:**

```
sp_rename 'GetCustomerSalary', 'newProcedure';
```

- **To Modify a Stored Procedure:**

```
ALTER PROCEDURE procedure_name
    AS
BEGIN
    -- New procedure code goes here
END
```

Pl/SQL : Stored Procedure

- **Deleting procedure:**
- **Syntax:**

DROP PROCEDURE [*IF EXISTS*] procedure_name;

Pl/SQL : Stored Function

- **Function:**

- It is a logically grouped set of PL/SQL & SQL statements that perform a specific task.

- It is a named pl/sql block that can accept parameters/ arguments & returns a value.

- It also consist of declarative part, executable part & optional exception handling part.

- Includes In, out and In out parameters.

- It is same as procedure except function returns single value.
- It avoids duplication of code.
- Provides security.

Pl/SQL : Stored Function

- **Function:**
- **Syntax:**

```
Create or replace function function_name   [parameters]
                return datatype is
                        -- Variable declarations;
                 Begin
                        -- executable code;
                 Exception
                         --Error handling code;
                  End
```

Pl/SQL : Stored Function

- **Function:**
- **Example: 1**

```
Create function Fun_emp
                    Return number is
                    Myempno number(4);
    Begin
            Select empno into myempno from emp;
            Return myempno;
    End;
```

Example: 2

```
    Create function f1 (x out number)
                Return number
                        Is
                Begin
                        x:= 10
                        Return x * x
                 End;
```

Pl/SQL : Stored Function

- **How to use Function in pl/sql?**

- **Example 3: with in argument**

```
Create function Rmt(x in number)
        Return number is
    Begin
        dbms_output. Put_line(x * x);
        return (x);
     End;
```

Calling a function:

```
    Begin
        dbms_output. Put_line(rmt(3));
    End;
```

Output: **3 * 3= 9**

Pl/SQL : Trigger

- **Trigger:**

- Block structure or stored procedure which is automatically fired when DML statements like insert, update & delete is executed on table or view.

- Each trigger is always associated with a table.

- which is activated on any DML statement such as **INSERT, UPDATE**, or **DELETE.**

- A trigger is called a special procedure because it cannot be called directly like a stored procedure.

Pl/SQL : Trigger

- **Trigger Example:**
- The UPDATE statement will update an existing record in the table –

UPDATE customers SET salary = salary + 500 WHERE id = 2;

- When a record is updated in the CUSTOMERS table,
- the above create trigger, **display_salary_changes** will be fired and it will display the following result

- Old salary: 1500
- New salary: 2000
- Salary difference: 500

Pl/SQL : Trigger

- **Trigger Types:**

Pl/SQL : Trigger

- **Trigger:**

- The main difference between the trigger and procedure is that a trigger is called automatically when a data modification event is made against a table.

- In contrast, a stored procedure must be called explicitly.

```
CREATE TRIGGER trigger_name
   (AFTER | BEFORE) (INSERT | UPDATE | DELETE)
         ON table_name
          FOR EACH ROW
     BEGIN
          --variable declarations
          --trigger code
     END;
```

Pl/SQL : Trigger

Oracle trigger:

CREATE [OR REPLACE] **TRIGGER** trigger_name
(**AFTER** | BEFORE) (**INSERT** | **UPDATE** | **DELETE**)
ON table_name
[**FOR** EACH ROW /For each statement]
[WHEN condition]
DECLARE
-- variable declarations
BEGIN
-- trigger code
EXCEPTION
-- exception handling
END;

Pl/SQL : Trigger

- **Trigger:**
- ***BEFORE | AFTER| INSTEAD OF:***
- The BEFORE or AFTER option specifies when the trigger fires, either before or after a triggering event e.g., INSERT, UPDATE, or DELETE.

- INSTEAD OF clause is used for creating trigger on a view.

- ON table_name: The table_name is the name of the table associated with the trigger.

- [FOR EACH ROW] – This specifies a row-level trigger, i.e., the trigger will be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.

- WHEN (condition) – This provides a condition for rows for which the trigger would fire. This clause is valid only for row-level triggers

Pl/SQL : Trigger

- **Trigger Example 1:Create Before insert trigger**

```
Create trigger trg1
      Before
    Insert on Emp
       For each row
Begin
          Dbms_output.put_line('you are about inserting values');
End;
```

Example 2: Create After insert trigger

```
Create trigger trg2
      After
    Insert on Emp
       For each row
Begin
          Dbms_output.put_line('you are about inserting values');
End;
```

Pl/SQL : Trigger

- **Trigger Example:**

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	30	Pune	2000
2	Riyansh	33	Mumbai	1500
3	Abhay	23	Latur	2000
4	Chaitali	34	Kolhapur	6500
5	Hardik	22	solapur	3000

- creates a **row-level** trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table.

- This trigger will display the salary difference between the old values and new values –

Pl/SQL : Trigger

- **Trigger Example:**

```
CREATE OR REPLACE TRIGGER display_salary_changes
        BEFORE DELETE OR INSERT OR UPDATE
                ON customers
                FOR EACH ROW
                WHEN (NEW.ID > 0)
DECLARE
        sal_diff number;
BEGIN
        sal_diff := :NEW.salary - :OLD.salary;
        dbms_output.put_line('Old salary: ' || :OLD.salary);
        dbms_output.put_line('New salary: ' || :NEW.salary);
        dbms_output.put_line('Salary difference: ' || sal_diff);
END; /
```

Pl/SQL : Trigger

- **Trigger Example:**
- The UPDATE statement will update an existing record in the table –

UPDATE customers SET salary = salary + 500 WHERE id = 2;

- When a record is updated in the CUSTOMERS table,
- the above create trigger, **display_salary_changes** will be fired and it will display the following result

- Old salary: 1500
- New salary: 2000
- Salary difference: 500

Difference between Function and Stored procedure

Function	Stored Procedure
Always returns a single value; either scalar or a table.	Can return zero, single or multiple values.
Functions are compiled and executed at run time.	Stored procedures are stored in parsed and compiled state in the database.
Only Select statements. DML statements like update & insert are not allowed.	Can perform any operation on database objects including select and DML statements.
Allows only input parameters. Does not allow output parameters.	Allows both input and output parameters
Does not allow the use of Try…Catch blocks for exception handling.	Allows use of Try…Catch blocks for exception handling.
Cannot have transactions within a function.	Can have transactions within a stored procedure.

Difference between Function and Stored procedure

Function	Stored Procedure
Cannot call a stored procedure from a function.	Can call a function from a stored procedure.
Temporary tables cannot be used within a function. Only table variables can be used.	Both table variables and temporary tables can be used.
Functions can be called from a Select statement.	Stored procedures cannot be called from a Select/Where or Having statements. Execute statement has to be used to execute a stored procedure.
Functions can be used in JOIN clauses.	Stored procedures cannot be used in JOIN clauses

```
DECLARE
  c_id customers.id%type := 8;
  c_name  customers.name%type;
  c_addr customers.address%type;
BEGIN
  SELECT  name, address INTO  c_name, c_addr
  FROM customers
  WHERE id = c_id;
DBMS_OUTPUT.PUT_LINE ('Name: '||  c_name);
 DBMS_OUTPUT.PUT_LINE ('Address: ' || c_addr);
EXCEPTION
  WHEN no_data_found THEN
    dbms_output.put_line('No such customer!');
  WHEN others THEN
    dbms_output.put_line('Error!');
END;
/
```

Pl/SQL : Exception handling

- **Exception handling :**

- An error occurs during the program execution is called Exception in PL/SQL.

- An error condition during program execution is called an exception, and the mechanism for resolving such an exception is known as exception handling.

- PL/SQL facilitates programmers to catch such conditions using exception block in the program and an appropriate action is taken against the error condition.

- There are two type of exceptions:

- System-defined Exceptions
- User-defined Exceptions

Pl/SQL : Exception handling

- **Example:**

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling goes here >
  WHEN exception1 THEN
     exception1-handling-statements
  WHEN exception2  THEN
    exception2-handling-statements
  WHEN exception3 THEN
    exception3-handling-statements
  ........
  WHEN others THEN
    exception-handling-statements
END;
```

Pl/SQL : Exception handling

- **Example:**

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling goes here >
  WHEN exception1 THEN
     exception1-handling-statements
  WHEN exception2  THEN
    exception2-handling-statements
  WHEN exception3 THEN
    exception3-handling-statements
  ........
  WHEN others THEN
    exception-handling-statements
END;
```

Pl/SQL : Exception handling

- **Example:**

```
DECLARE
        c_id customers.id%type := 8;
        c_name  customers.name%type;
        c_addr customers.address%type;
BEGIN
   SELECT  name, address INTO  c_name, c_addr  FROM customers  WHERE id = c_
    id;
            DBMS_OUTPUT.PUT_LINE ('Name: '||  c_name);
            DBMS_OUTPUT.PUT_LINE ('Address: ' || c_addr);
EXCEPTION
        WHEN no_data_found THEN
                dbms_output.put_line('No such customer/ no data found!');
        WHEN others THEN
                dbms_output.put_line('Error!');
END;    /
```

Pl/SQL : Cursor

- A **Cursor** is a *temporary memory* that is allocated by the database server at the time of performing the **DML** operations such as INSERT, UPDATE and DELETE etc., on a table.
- It holds the multiple rows returned by the SQL statement.
- You can use it to retrieve and manipulate data stored in the SQL tables.
- Oracle creates a *memory area*, known as the *context area*, for processing an SQL statement, which contains all the information needed for processing the statement;

Pl/SQL : Cursor

- Example, the number of rows processed, etc.
- A cursor is a *pointer to this context area*.
- PL/SQL controls the *context area through a cursor*.
- A cursor holds the rows (one or more) returned by a SQL statement.
- The set of rows the cursor holds is referred to as the active set.
- There are two types of cursors –
  - Implicit cursors
  - Explicit cursors

Pl/SQL : Cursor

- **Implicit Cursors:**

- Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement.

- Programmers cannot control the implicit cursors and the information in it.

- Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.

- For *INSERT operations*, the cursor holds the data that needs to be inserted.

- For *UPDATE* and *DELETE* operations, the cursor identifies the rows that would be affected.

Pl/SQL : Cursor

- SQL cursor **Attributes** such as:

- **%FOUND,**
- **%ISOPEN,**
- **%NOTFOUND**, and
- **%ROWCOUNT**.

- SQL cursor attribute will be accessed as ***sql%attribute_name***

- The SQL cursor has **additional attributes:**

- **%BULK_ROWCOUNT** and
- **%BULK_EXCEPTIONS**,

- designed for use with the **FORALL** statement.

Pl/SQL : Cursor

- **%FOUND**
- Returns *TRUE* if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows.
- Otherwise, it returns *FALSE*.

- **%NOTFOUND**
- The logical opposite of %FOUND. It returns *TRUE* if an INSERT, UPDATE, or DELETE statement affected *no rows*, or a SELECT INTO statement returned *no rows*. Otherwise, it returns ***FALSE***.

- **%ISOPEN**
- Always ***returns FALSE*** for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.

- **%ROWCOUNT**
- Returns the *number of rows affected* by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Pl/SQL : Cursor

- **Implicit Cursors Example:**
- The following program will update the table and increase the salary of each customer by 500 and use the **SQL%ROWCOUNT** *attribute to determine the number of rows affected* –

```
DECLARE total_rows number(2);
BEGIN
        UPDATE customers SET salary = salary + 500;
  IF sql%notfound
     THEN
        dbms_output.put_line('no customers selected');
  ELSIF
        sql%found
THEN
        total_rows := sql%rowcount;
       dbms_output.put_line( total_rows || ' customers selected ');
END IF;    END; /
```

Pl/SQL : Cursor

- **Implicit Cursors:**
- **Output:**
- 6 customers selected

Pl/SQL : Cursor

- **Explicit Cursors:**
- Explicit cursors are *programmer-defined cursors* for gaining more control over the **context area**.

- It should be *defined in the declaration section* of the PL/SQL Block.

- It is created on a *SELECT Statement* which returns more than one row.

- ***Working with an explicit cursor includes the following steps*** –

- **Declaring** the cursor for ***initializing*** the memory
- **Opening** the cursor for ***allocating*** the memory
- **Fetching** the cursor for ***retrieving*** the data
- **Closing** the cursor to ***release the allocated memory***

Pl/SQL : Cursor

- **Explicit Cursors:**

✓ **Declaring the Cursor:**

- Declaring the cursor defines the cursor with a name and the associated SELECT statement.
- **Syntax:**

  CURSOR cursor_name **IS** SELECT statement;

- **Ex.:**

  CURSOR c_customers **IS** *SELECT id, name, address FROM customers;*

✓ **Opening the Cursor:**

- Opening the cursor *allocates the memory for the cursor* and *makes it ready for fetching the row*s returned by the SQL statement into it.

- **Syntax:** **OPEN** cursor_name;

- **Example:** OPEN **c_customers**;

Pl/SQL : Cursor

- **Explicit Cursors:**

✓ **Fetching the Cursor:**

- Fetching the cursor involves accessing one row at a time. For example, we will fetch rows from the above-opened cursor as follows –

- **Syntax**: **FETCH** cursor_name **INTO** variable_list;

- **Example**: ***FETCH*** c_customers INTO c_id, c_name, c_addr;

✓ **Closing the Cursor:**

- Closing the cursor means releasing the allocated memory.

- **Syntax:** ***Close*** cursor_name;
- **Example**: ***CLOSE*** c_customers;

Pl/SQL : Cursor

Explicit Cursors: Execute the following program to retrieve the customer name and address.

```
DECLARE
  c_id customers.id%type;
  c_name customers.name%type;
  c_addr customers.address%type;
  CURSOR c_customers is
    SELECT id, name, address FROM customers;
BEGIN
  OPEN c_customers;
  LOOP
    FETCH c_customers into c_id, c_name, c_addr;
    EXIT WHEN c_customers%notfound;
    dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
  END LOOP;
  CLOSE c_customers;
END;
```

Pl/SQL : Function

- **Execution:**

Triggers	Stored Procedures
A special kind of stored procedure that is not called directly by a user. In fact, a trigger is created and is programmed to fire when a specific event occurs.	A group of SQL statements which can be reused again and again. These statements are created and stored in the database.
A trigger cannot be called or execute directly by a user. Only when the corresponding events are fired, triggers are created.	Can execute stored procedures by using the exec command, whenever we want.
You cannot schedule a trigger.	You can schedule a job to execute the stored procedure on a pre-defined time.
Cannot directly call another trigger within a trigger.	Call a stored procedure from another stored procedure.
Parameters cannot be passed as input	Parameters can be passed as input
Cannot return values.	Can return zero or n values.
Transactions are not allowed within a trigger.	You can use transactions within a stored procedure.

Pl/SQL : Stored Procedure

- **Execution:**

Clustered Index	Non-clustered Index
A clustered index is faster	Non clustered index is relatively slower
Alters the way records are stored in a database as it sorts out rows by the column which is set to be clustered index	Does not alter the way it was stored but it creates a separate object within a table which points back to the original table rows after searching
One table can only have one clustered index	One table can only have many non clustered indexes

DROP	TRUNCATE	DELETE
Used to delete a database, table or a view	Used to delete all rows from a table	Used to delete a row in the table
Data cannot be rollbacked	Data cannot be rollbacked	Data can be rollbacked
A DDL command	A DDL command	A DML command.
Slower than TRUNCATE	Faster than DROP and DELETE	Slower than TRUNCATE
Deletes the full structure of the table	Preserves the structure of the table	Deletes the structure of the row from a table

HAVING	WHERE
Used only with SELECT statement	Used in a GROUP BY clause
Used with the GROUP BY function in a query	Applied to each row before they are a part of the GROUP BY function in a query

4. Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory.

- Suggested Problem statement:
- Consider Tables:
- 1. Borrower(Roll_no, Name, DateofIssue, NameofBook, Status)
- 2. Fine(Roll_no,Date,Amt)
- Accept Roll_no and NameofBook from user.
- Check the number of days (from date of issue).
- If days are between 15 to 30 then fine amount will be Rs 5per day.
- If no. of days>30, per day fine will be Rs 50 per day and for days less than 30, Rs. 5 per day.
- After submitting the book, status will change from I to R.
- If condition of fine is true, then details will be stored into fine table.
- Also handles the exception by named exception handler or user define exception handler.

- **OR**
- Write a PL/SQL code block to calculate the area of a circle for a value of radius varying from 5 to 9. Store the radius and the corresponding values of calculated area in an empty table named areas, consisting of two columns, radius and area.

- Create procedure finecal(in roll_no int, in book_name char(20))
- Begin
- Declare
- v_doi date;
- noofdays int ;
- fine double;
- f1 int default 0;
- Declare continue handler for not found set f1=1;
- Select Date_of_issue into v_doi from borrower where Roll_no= roll_no and Name_of_Book=book_name;
- If f1=1 then
- Select “Record not found”;
- Else
- Set noofdays :=datediff(sysdate(),v_doi);
- If (noofdays >=15 and noofdays <=30) then
- Set fine:=((noofdays -30)*50) + 150;

- Else
- Set fine:=0;
- End if;
- Select fine;
- Update borrower set status:='R' where Roll_no=roll_no and Name_of_Book=book_name;
- Insert into fine values (Roll_no, sysdate(), fine);
- End if ;
- End;

- Call procedure:
- Call finecal(14, "oop");

- Borrower(Roll_no, Name, DOI, Book_name,status)
- Fine(Roll_no, Date, amount)

- amount double
- Borrower(Roll_no, Name, DOI, Book_name, status)

Roll_no	Name	DOI	Book_name	status
8	Raj	2020-11-05	DSA	I
11	Monika	2021-02-14	CNS	I
14	Riyansh	2020-09-1	OOP	I
16	Eva	2021-01-26	MP	I
20	aditya	2020-08-1	TOC	I

- **Named PL/SQL Block: PL/SQL Stored Procedure and Stored Function.**
- Write a Stored Procedure namely proc_Grade for the categorization of student. If marks scored by students in examination is <=1500 and marks>=990 then student will be placed in distinction category if marks scored are between 989 and 900 category is first class, if marks 899 and 825 category is Higher Second Class.
- Write a PL/SQL block to use procedure created with above requirement.
- Stud_Marks(name, total_marks) Result(Roll,Name, Class)

- **Database Trigger (All Types: Row level and Statement level triggers, Before and After Triggers).**
- Write a database trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_Audit table.

- Create table book(accno int, author varchar(20),publisher varchar(20),count int)

- Create table lib_audit(accno int, author varchar(20),publisher varchar(20),count int, action varchar(20))

After delete Trigger

```
Create trigger after_book_delete
After delete ON book
For each row
Begin
Insert into lib_audit  set action =‘delete’,
accno=OLD.accno,
title=OLD.title,
author=OLD.author,
publisher=OLD.publisher,
count=OLD.count;
END//
```

Before update trigger

```
Create trigger before_book_update
Before  update ON book
For each row
Begin
Insert into lib_audit  set action ='update',
accno=New.accno,
title=New.title,
author=New.author,
publisher=New.publisher,
count=New.count;
END//
```